



Compatibility Automation Platform

Architecture & Developer Guide

Generic · Auditable · Deterministic · VS Code Copilot Integrated

Table of Contents

1.	Executive Summary
2.	System Architecture Overview
3.	Pipeline — Step by Step
4.	Generic Configuration
5.	Version Constraint Types
6.	Safety & Gating Policy
7.	Human Review with VS Code Copilot
8.	File Layout Reference
9.	Query CLI Reference
10.	Adding a New Product
11.	Script Reference
12.	GitHub Actions Workflow

1. Executive Summary

The Compatibility Automation Platform is a **product-agnostic**, **deterministic**, and **auditable** system for tracking whether a given software product version is certified to run on a specific platform version. It replaces manual spreadsheet tracking with an automated pipeline that:

- Fetches vendor compatibility documents (HCLs, release notes, advisories) automatically.
- Extracts the exact supported version range using conservative, deterministic rules — no inference.
- Compares every change against a versioned baseline and classifies it as safe (narrowing) or unsafe (widening/ambiguous).
- Answers instant queries: *"Is veritas-infoscale 8.0.2 compatible with RHEL 8 kernel 4.18.0-305.el8?"*
- Integrates VS Code GitHub Copilot for human-in-the-loop review when automation is not confident — no external API keys required.
- Runs end-to-end in GitHub Actions on a weekly schedule, with per-product matrix parallelism.

Metric	Value
Supported version types	kernel_range · semver · package_version
AI dependency	VS Code GitHub Copilot (no API keys needed)
Auto-merge safety	Widening changes always blocked
Audit trail	SHA-256 hashes + provenance JSON per file
CI cadence	Weekly (Monday 03:00 UTC) + manual dispatch
Adding a new product	One YAML config file

2. System Architecture Overview

The system is organized around a **config ID** — a slug that identifies a product+platform pair (e.g., `veritas-infoscale_rhel`). Every file in the system is namespaced under this ID, which means multiple products and platforms can coexist with zero interference.

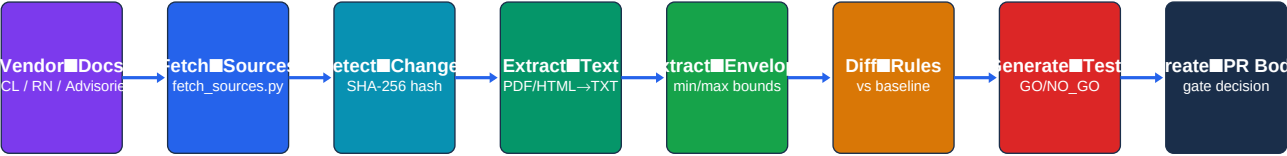


Figure 1 — Main automation pipeline

The pipeline has two execution paths depending on whether automated extraction succeeds with high confidence:

Path	Trigger	Outcome
Automated (happy path)	Docs contain explicit version bounds with support phrase <code>Rule updated</code>	Rules updated, tests generated, PR created automatically
Human review (Copilot)	No docs, ambiguous extraction, or confidence < high	Review prompt generated → user + Copilot fill template → pipeline

3. Pipeline — Step by Step

Step 1: `fetch_sources.py`

Downloads vendor documents from URLs listed in `configs/sources/{id}.yaml` into `data/{id}/raw/`. Writes a `.provenance.json` sidecar per file recording the URL, download timestamp, and SHA-256 hash. Supports local file copy (`local_path`) and HTTP download.

```
python3 scripts/fetch_sources.py --config-id veritas-infoscale_rhel
```

Step 2: `detect_changes.py`

Hashes every file in `data/{id}/raw/` using SHA-256 and compares against the previous run's state in `ingest_state.json`. Produces `pending.json` — the list of files that changed and must be re-extracted. Unchanged files are skipped.

```
python3 scripts/detect_changes.py --config-id veritas-infoscale_rhel
```

Step 3: `extract_text.py`

Converts each pending raw file to plain text in `data/{id}/parsed/`. TXT files are copied directly. PDF and HTML are currently stubbed with an `EXTRACTION_PENDING` marker — add parsers (pdfminer, BeautifulSoup) as needed. Updates `manifest.json` with `raw→parsed` mappings.

```
python3 scripts/extract_text.py --config-id veritas-infoscale_rhel
```

Step 4: `extract_envelope.py`

The core extraction step. Applies the `version_regex` from the config to each parsed text file, looking for lines that contain both a support phrase (*support / certified / compatible*) AND two or more version strings. Confidence is **high** when both conditions are met on the same line, **medium** when versions are found without the phrase, and **low** when nothing is found. Writes the result to `rules/generated/{id}.yaml` and prints `HUMAN_REVIEW_NEEDED` if confidence is not high.

```
python3 scripts/extract_envelope.py --config-id veritas-infoscale_rhel
```

Step 5: `diff_rules.py`

Loads the generated rule and the baseline rule and compares their version bounds using `VersionComparator` (tuple comparison — not string comparison). Classifies the change as widening, narrowing, or ambiguous. Also checks confidence, vendor source presence, and ambiguity flag. Writes `diff_summary.json`.

```
python3 scripts/diff_rules.py --config-id veritas-infoscale_rhel
```

Step 6: `generate_tests.py`

Creates GO and NO_GO test cases from the generated rule's bounds. Uses `VersionComparator.bump()` to produce an above-max version for the NO_GO case — this works generically for any version format. Writes to `tests/generated/{id}_cases.yaml`.

```
python3 scripts/generate_tests.py --config-id veritas-infoscale_rhel
```

Step 7: `create_pr.py`

Reads `diff_summary.json` and produces a Markdown PR body with a gating decision table. The gate is **pr_block_auto_merge** if any config has a blocking decision, **pr_allowed** if all are narrowing with high confidence, or **info** for metadata-only changes. Does not push to GitHub directly — wire to gh CLI in CI.

```
python3 scripts/create_pr.py --config-id veritas-infoscale_rhel
```

4. Generic Configuration

Each product+platform pair is described by a single YAML config file. The entire pipeline reads from this config — no hardcoded product names, version strings, or regex patterns exist in the scripts.

Source Config (configs/sources/{id}.yaml)

```
id: veritas-infoscale_rhel product: name: veritas-infoscale version: "8.0.2" platform: name: rhel
version: "8" version_constraint: type: kernel_range version_regex: '\d+\.\d+\.\d+-(\d+)\.el\d+'
comparison_groups: [1] # only the build number matters for ordering sources: - name:
veritas_hcl_oct2023 url: "" # URL to download type: vendor_hcl local_path: "" # or path to local file
```

Baseline Rule (rules/baselines/{id}.yaml — schema v2.0)

```
schema_version: "2.0" id: veritas-infoscale_rhel product: {name: veritas-infoscale, version: "8.0.2"}
platform: {name: rhel, version: "8"} compatibility: status: supported version_constraint: type:
kernel_range min: 4.18.0-193.el8 max: 4.18.0-425.el8 unsupported_examples: [4.18.0-80.el8,
4.18.0-477.el8] notes: risk: "Storage/clustering instability outside envelope" remediation: "Stay
within envelope or get vendor confirmation" metadata: sources: [{name: Veritas HCL Oct 2023, type:
vendor_hcl}] confidence: medium # high | medium | low | unknown owner: sre-team last_reviewed:
"2026-01-30" review_interval_days: 90 ambiguous: false
```

5. Version Constraint Types

The `VersionComparator` class in `scripts/lib/version_compare.py` provides generic version parsing, comparison, and bumping. It is driven entirely by the config's `version_constraint` block — no product-specific code exists in the scripts.

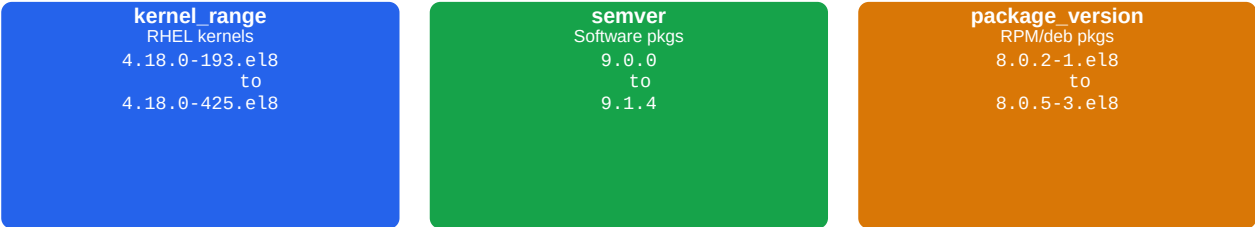


Figure 2 — The three supported version constraint types

Type	version_regex example	comparison_groups	Used for
kernel_range	<code>\d+\.\d+\.\d+-(\d+)\.el\d+</code>	[1]	RHEL kernel build number
semver	<code>(\d+)\.(\d+)\.(\d+)</code>	[1, 2, 3]	Standard semantic versions
package_version	<code>(\d+)\.(\d+)\.(\d+)-(\d+)</code>	[1,2,3,4]	RPM/deb package releases

How VersionComparator works

All comparison reduces to comparing tuples of integers. The regex captures groups, and `comparison_groups` selects which groups to use for ordering. For RHEL kernels, only group 1 (the build ID after the dash) is used because the `4.18.0` prefix and `el8` suffix are invariant within a platform version.

```
# kernel 4.18.0-425.el8 → parse → (425,) # kernel 4.18.0-477.el8 → parse → (477,) #
is_within("4.18.0-305.el8", "4.18.0-193.el8", "4.18.0-425.el8") # (305,) within [(193,), (425,)] →
True → COMPATIBLE # bump("4.18.0-425.el8") → "4.18.0-426.el8" (for NO_GO test case)
```

6. Safety & Gating Policy

The system is conservative by design: any change that could increase risk is automatically blocked. Only changes that reduce risk (narrowing) or are purely informational can proceed without human approval.

Change Type	Condition	Decision	Label
Envelope widening	Lower min OR higher max than baseline	BLOCKED	auto-merge-blocked
Ambiguous extraction	No explicit bounds found in docs	BLOCKED	auto-merge-blocked
Low confidence	Confidence < high after extraction	BLOCKED	auto-merge-blocked
Missing vendor source	No vendor_hcl / release_notes / advisory	BLOCKED	auto-merge-blocked
Envelope narrowing	Higher min AND lower max than baseline	ALLOWED	auto-merge-allowed
Metadata-only change	Bounds unchanged, other fields differ	INFO	info
No change	Identical to existing baseline	SKIP	skip

Figure 3 — Gating decision matrix

Confidence levels

Level	Condition	Auto-merge?
high	Support phrase + 2 version strings on the same line	Narrowing only
medium	2 version strings found but no explicit support phrase	No — blocked
low	Fewer than 2 version strings found in any document	No — blocked
unknown	No documents processed or rule not yet established	No — blocked

7. Human Review with VS Code Copilot

When automation cannot extract version bounds with high confidence, the system falls back to a structured human review loop. No external AI API keys are required — the user pastes a generated prompt into **VS Code GitHub Copilot Chat**.

Human Review Loop (VS Code Copilot Integration)

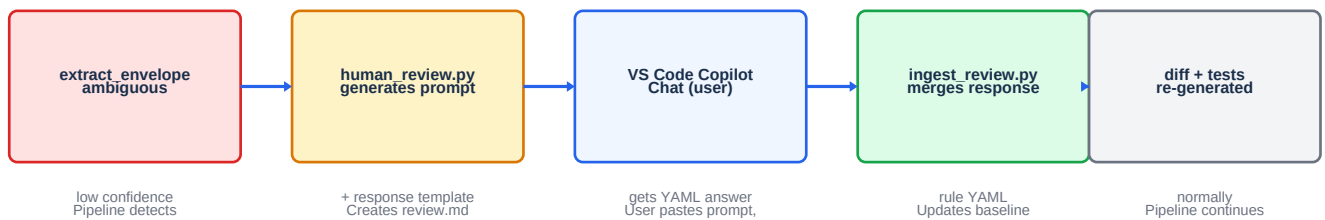


Figure 4 — Human review loop using VS Code Copilot

Step-by-step human review

- 1 Pipeline detects ambiguity**
extract_envelope.py prints HUMAN_REVIEW_NEEDED to stdout. GitHub Actions detects this and runs human_review.py automatically.
- 2 Review files generated**
human_review.py creates two files in human_reviews/{id}/: • YYYY-MM-DD_review.md — the Copilot Chat prompt to paste • YYYY-MM-DD_response_template.yaml — pre-filled template to complete
- 3 User opens in VS Code**
Open the review.md file. Open Copilot Chat (Ctrl+Shift+I). Optionally attach the vendor PDF/HTML. Paste the prompt and send.
- 4 Fill the response template**
Copy Copilot's YAML output into the response_template.yaml. The template already has current baseline values as defaults — only override what Copilot provides.
- 5 Ingest the response**
Run ingest_review.py --config-id --review . This deep-merges the response into the baseline, sets ambiguous=false, updates last_reviewed, and re-runs diff + test generation.
- 6 Pipeline resumes**
create_pr.py picks up the updated diff_summary.json and produces a PR body with the corrected gating decision.

8. File Layout Reference

```

configs/sources/

    {id}.yaml

data/{id}/

    raw/

    parsed/

rules/

    baselines/{id}.yaml

    generated/{id}.yaml

tests/

    baselines/{id}_cases.yaml

    generated/{id}_cases.yaml

human_reviews/{id}/

    YYYY-MM-DD_review.md

    YYYY-MM-DD_response*.yaml

scripts/

    lib/  query.py  human_review.py ...

.vscode/tasks.json

.github/copilot-instructions.md

```

Figure 5 — Repository file layout

Path pattern	Description
configs/sources/{id}.yaml	Product+platform config: version_regex, sources, notes
data/{id}/raw/	Downloaded vendor docs (PDF, HTML, TXT)
data/{id}/parsed/	Extracted text, ingest_state.json, pending.json, manifest.json
rules/baselines/{id}.yaml	Authoritative baseline rule (schema v2.0, manually maintained)
rules/generated/{id}.yaml	Auto-extracted rule produced by extract_envelope.py
tests/baselines/{id}_cases.yaml	Hand-written GO/NO_GO test cases
tests/generated/{id}_cases.yaml	Auto-generated test cases from version bounds
human_reviews/{id}/	Copilot review prompts and response templates
scripts/lib/version_compare.py	VersionComparator: parse, is_within, bump, compare
scripts/lib/config_loader.py	load_config(), list_config_ids(), validation
scripts/lib/path_resolver.py	All file paths — single source of truth
scripts/query.py	CLI query: exit 0=COMPATIBLE 1=NOT_COMPATIBLE 2=UNKNOWN
scripts/human_review.py	Generate Copilot review prompt + response template

scripts/ingest_review.py	Merge Copilot response into baseline, re-run diff+tests
.vscode/tasks.json	5 VS Code tasks: query, pipeline, open reviews
.github/copilot-instructions.md	Copilot context: schemas, rules, expected output format

9. Query CLI Reference

`scripts/query.py` is the primary user-facing feature. It reads all baseline rules and answers compatibility questions in under a second.

Basic query

```
python3 scripts/query.py \ --product veritas-infoscale \ --platform rhel \ --version 4.18.0-305.el8
```

Scoped to specific versions

```
python3 scripts/query.py \ --product veritas-infoscale --product-version 8.0.2 \ --platform rhel  
--platform-version 8 \ --version 4.18.0-477.el8
```

JSON output (for scripting)

```
python3 scripts/query.py --product veritas-infoscale \ --platform rhel --version 4.18.0-305.el8  
--json
```

Exit code	Meaning	Use case
0	COMPATIBLE	Version is within the documented supported range
1	NOT_COMPATIBLE	Version is outside the range or product is unsupported
2	UNKNOWN	No rule found — run the pipeline or add a config

10. Adding a New Product

Adding support for a new product+platform pair requires **one config file** and optionally a baseline rule. No script changes are needed — the pipeline discovers new configs automatically.

1. Create `configs/sources/{new-id}.yaml` with product, platform, version_regex, and sources.
2. Optionally create `rules/baselines/{new-id}.yaml` if you already know the bounds.
3. Add "{new-id}" to the options array in `.vscode/tasks.json`.
4. Run `python3 scripts/fetch_sources.py --config-id {new-id}` to download vendor docs.
5. Run the full pipeline. If extraction is ambiguous, run `human_review.py` and use Copilot.
6. Commit `configs/sources/{new-id}.yaml` and `rules/baselines/{new-id}.yaml`.
7. GitHub Actions will pick up the new config on the next scheduled run automatically.

Example: Oracle DB 19c on RHEL 8

```
id: oracle-db_rhel product: name: oracle-db version: "19c" platform: name: rhel version: "8"
version_constraint: type: semver version_regex: '(\d+)\.(\d+)\.(\d+)' comparison_groups: [1, 2, 3]
sources: - name: oracle_cert_matrix_2026 url: "" # paste URL when available type: vendor_hcl
```

11. Script Reference

Script	Role	Description
query.py	Query	Answer: is product X version Y compatible? Args: --product --platform --version [--product-version] [--platform-version] [--json]
fetch_sources.py	Fetch	Download vendor docs from URLs in config. Args: [--config-id]
detect_changes.py	Detect	Hash raw files, produce pending.json. Args: [--config-id]
extract_text.py	Extract	Convert raw files to plain text. Args: [--config-id]
extract_envelope.py	Envelope	Parse min/max version bounds from text. Prints HUMAN_REVIEW_NEEDED if ambiguous. Args: [--config-id]
diff_rules.py	Diff	Compare generated vs baseline using VersionComparator. Write diff_summary.json. Args: [--config-id]
generate_tests.py	Tests	Build GO/NO_GO test YAML using VersionComparator.bump(). Args: [--config-id]
create_pr.py	PR	Produce pr_body.md with gating table. Does not call GitHub API. Args: [--config-id]
human_review.py	Review	Generate Copilot Chat prompt + response template in human_reviews/{id}/. Args: --config-id
ingest_review.py	Ingest	Deep-merge Copilot response into baseline. Re-runs diff + test generation. Args: --config-id --review <path>
migrate_legacy.py	Migrate	One-time: convert old docs/ structure to new data/ layout. Run once.

12. GitHub Actions Workflow

The workflow at `.github/workflows/compatibility-update.yml` runs on a weekly schedule and on manual dispatch. It uses a **dynamic matrix strategy** — the list of configs is read at runtime from `configs/sources/*.yaml`, so adding a new product requires no workflow changes.

Job structure

Job	Purpose
setup	Reads <code>configs/sources/*.yaml</code> , builds matrix of config IDs
compatibility-update	Runs once per config ID (parallel, fail-fast: false)

Per-config steps

Step	Detail
Checkout + Python setup	<code>actions/checkout@v4</code> , <code>setup-python@v5</code> (3.11)
Install deps	<code>pip install pyyaml requests</code>
<code>fetch_sources.py</code>	Download vendor docs
<code>detect_changes.py</code>	Hash and queue changed files
<code>extract_text.py</code>	Convert to plain text
<code>extract_envelope.py</code>	Parse bounds; set <code>needs_review=true</code> if <code>HUMAN_REVIEW_NEEDED</code> in stdout
<code>human_review.py</code> (conditional)	Only runs if <code>needs_review=true</code> ; uploads review files as artifact
<code>diff_rules.py</code>	Compare to baseline
<code>generate_tests.py</code>	Generate test cases
<code>create_pr.py</code>	Produce PR body
Upload artifacts	<code>rules/generated/</code> , <code>tests/generated/</code> , <code>data/{id}/parsed/</code> , <code>human_reviews/{id}/</code>

Manual dispatch

Go to **Actions** → **Compatibility Update** → **Run workflow** and enter a config ID (e.g., `veritas-infoscale_rhel`) to run a single config. Leave blank to run all configs.

For questions or contributions, see the repository README or open a GitHub issue.